

Composition Pair: Path Retracementability × Determinism Contract — How Two Foundational Commitments Compose to Produce Reproducibility in CKS Substrates

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 6, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the architectural property that emerges when CKS’s path retraceability commitment composes with its determinism contract — a property neither commitment yields in isolation — so that downstream implementers, auditors, and critics have a single citable specification of what reproducibility means in a CKS-coherent substrate.

Abstract

The CKS pattern commits to path retraceability — the structural property that every piece of substrate content carries enough provenance to reconstruct its causal antecedents from substrate content alone — and to a determinism contract specifying five guarantees about how substrate state behaves under reads, writes, conflict-handling, and bounded non-determinism. Both commitments are load-bearing in the source paper. Neither, in isolation, suffices to make substrate state reproducible — re-derivable from a known prior state and the recorded sequence of rule applications. Path retraceability alone admits provenance records whose replay produces different results because cell behavior depends on hidden or implicit state. The determinism contract alone admits operations that behave consistently but leave no addressable record by which they can be replayed. The composition of the two is what produces *reproducibility* as an emergent architectural property: substrate state at any point in deployment lifecycle can be re-derived from a known prior state plus the sequence of provenance records, supporting audit verification, recovery, regression testing, and migration. This note states the four operational components of the emergent property, identifies what the composition forces beyond either commitment in isolation, names the anti-pattern classes that violate the composition specifically, distinguishes the property from four adjacent architectural patterns, and provides an operational test built around three sharpening properties — replay-fidelity, addressable-provenance, and deterministic-substrate-operation.

1. Why the composition needs to be formalized as standalone

Path retraceability and the determinism contract are each defended in the source paper as foundational commitments. Path retraceability is named in §3.1 as the structural requirement that every piece of substrate content carry enough provenance — writer, rule reference, timestamp, prior-state, change description, cell-execution-id — that the path back to its antecedents can be reconstructed from substrate content alone. The determinism contract is collected from §2.1, §3.1, §4.1, §6.2, §8.2, and §11.3 as five guarantees over substrate operations: read determinism, cell-behavior determinism, change addressability, conflict-handling determinism, and bounded non-determinism. Both commitments are operationally precise. Neither, in isolation, captures the architectural property §5 leans on when discussing how retraceability and determinism support audit and recovery — that a CKS substrate’s state can be replayed.

The replay property is what this note formalizes. It is the architectural pattern that lets humans verify substrate outcomes by re-executing recorded operations from a known prior state, lets deployments recover by re-applying recorded changes, lets regression tests run historical inputs under modified rules, and lets migrations re-execute with the same rules to produce equivalent state on new infrastructure. None of these capabilities is available from path retraceability alone — provenance can be complete and yet replay can produce different results when cell behavior depends on hidden state, time, or external dependencies. None is available from the determinism contract alone — operations can be deterministic and yet replay can be impossible because no addressable record of what occurred exists. Replay requires both, composed.

Naming the composition as a single derivation matters strategically: “auditable AI” and “reproducible AI systems” are dominant 2024–2026 commercial concerns, and the architectural pattern that makes substrate state reproducible is consequential prior art. It also matters expositively: §5 leans on the composition implicitly, and a downstream reader is free to satisfy either foundational commitment individually while failing what §5 actually depends on.

2. The emergent architectural property — reproducibility

The composition of path retraceability and the determinism contract produces *reproducibility* as an architectural property of substrate state. Reproducibility means that substrate state at any point in deployment lifecycle can be re-derived from a prior known substrate state plus the sequence of rule applications recorded in retraceable provenance. The property is operationally specified as four components, each grounded in a specific sub-commitment of the two foundational commitments.

(a) Provenance is sufficient to re-derive substrate state. The six provenance metadata fields the source paper requires of every substrate change — actor, rule reference, timestamp, prior-state, change description, cell-execution-id — together capture enough information to re-execute the change and reach the same resulting state. The two fields that bear the composition’s weight are *prior-state* (so replay starts from a known input) and *rule reference* (so replay applies the same operation). Without prior-state, replay has no starting point; without rule reference, replay has no operation to apply.

(b) Change addressability produces stable identifiers for retracing. Every substrate change has a stable address — typically incorporating cell-execution-id, timestamp, or sequence number — that can be referenced for replay. Stability matters: an address unique within a single execution but changing on replay supports retraceability without supporting reproducibility, because addresses recorded in a

downstream change cannot be resolved against the replayed history. The composition forces addresses that are deterministic functions of provenance fields and survive replay.

(c) Read determinism holds for re-applied reads. Substrate reads from a known prior state always produce the same result. Replay starts from the prior-state recorded in provenance, and reads of that state during replay are consistent with the reads the original execution performed. This is what makes the prior-state field load-bearing for replay rather than merely for audit.

(d) Cell-behavior determinism holds for re-applied cell executions. Cell behavior is determined entirely by substrate state and orchestration rule. Re-applying the recorded rule to the recorded prior state produces the same substrate write the original execution produced — equivalence judged at the substrate-write layer, not at the LLM-output layer. Cells, when re-executed against historical state, behave the same way they did the first time.

A deployment satisfying the composition exhibits all four components. A deployment that satisfies any three but fails one fails the composition: provenance without addressability cannot be referenced for replay; addressability without sufficient provenance cannot drive replay; deterministic operations without records cannot be replayed; recorded operations without deterministic re-execution produce different results.

3. What the composition forces beyond either commitment in isolation

Neither foundational commitment, by itself, requires what the composition together requires. Path retraceability alone is satisfiable by minimal provenance — actor and timestamp suffice to attribute a change. The composition forces the full six fields, with prior-state and rule reference specifically populated, because without them replay cannot be started or driven. The determinism contract alone is satisfiable by deterministic operations in nominal cases — cells that read substrate state and write substrate state behave consistently when run twice in close succession. The composition forces cell behavior to depend *only* on substrate state and rule, excluding hidden state, implicit context, time-of-day, randomness, and external state outside the bounded non-determinism categories the contract permits. The composition tightens the determinism guarantee from “operates deterministically when conditions are favorable” to “operates deterministically when re-executed against historical state.”

Several determinism sub-commitments are tightened in the same way. Change addressability is satisfiable on its own by addresses unique within an execution; the composition forces *stable* addressability — deterministic functions of provenance fields that survive replay. Conflict-handling determinism is satisfiable on its own by deterministic resolution at the moment of resolution; the composition forces resolution to be reproducible on replay, which means the conflict-handling rule operates over substrate state present in provenance, not over runtime state external to the substrate. Bounded non-determinism is satisfiable on its own by allowing LLM consultations whose outputs are non-deterministic but mediated by cells under rules; the composition tightens the boundedness requirement so that any non-determinism — LLM, randomness, time, external API — that affects substrate state is either recorded in provenance (so replay uses the recorded value) or excluded from cells whose substrate writes participate in retraceable history.

Provenance residency, as a sub-commitment of path retraceability, is satisfiable by recording provenance fields adjacent to substrate content. The composition forces provenance to live in substrate rather than in external audit logs because reproducibility is a substrate-only-path property: replay must be drivable from substrate content alone, without consulting external systems whose retention or availability is not under the substrate's governance.

4. Anti-patterns specifically violating the composition

A deployment can satisfy each foundational commitment individually and still fail the composition. The composition's distinct anti-pattern classes are what make it a separate derivation from either constituent.

Non-deterministic provenance. Provenance records exist and are complete, but cell behavior depends on hidden state — implicit context per A3.20, hidden state in cells per A3.16 — so re-applying the recorded rule to the recorded prior state produces a different substrate write than was originally produced. Path retraceability is satisfied; the determinism contract is violated through cell-behavior non-determinism; the composition fails because reproducibility requires both. This is the canonical composition violation.

Retraceability-without-addressability. Provenance fields are recorded, but the change lacks stable addressability — A3.19 (non-addressable writes) instantiates this failure. Replay cannot reference specific changes by stable identifier, and the sequence cannot be reconstructed from substrate content alone.

Determinism-without-provenance. Operations are deterministic, but no record of operations exists — provenance gaps prevent reconstruction of the sequence. Replay has nothing to replay.

Hidden state in provenance. Provenance records actor and rule, but cell behavior depended on hidden state not recorded in provenance. The recorded provenance is complete with respect to retraceability's required fields but insufficient for replay because the unrecorded hidden state determined the substrate write.

Conflict-handling non-determinism. Conflicts are resolved through implicit mechanisms — last-write-wins, vector clocks, ML-driven resolution — outside cell-mediated rule processing. A3.10 (contradiction collapse by automation) instantiates this failure. Replay of conflict-affected state produces different resolutions.

LLM consultation outside bounded non-determinism. LLM consultations affect substrate state without bounding — A3.12 (LLM-as-terminal-producer) instantiates this failure. Non-determinism propagates from LLM outputs to substrate state, breaking reproducibility because replay of the LLM consultation produces different outputs and therefore different substrate writes.

Time-, randomness-, and external-API-dependent cell behavior. Cells consult wall-clock time, current date, random number generators, or external APIs whose responses change over time, without recording the consulted values in provenance. Replay at a different time, with a different random seed, or against a changed external API produces different results. Each is a sub-class of unbounded non-determinism propagating to substrate state. The composition is satisfiable when the consulted

value is treated as bounded non-determinism — recorded in provenance so replay uses the recorded value rather than re-querying.

In each anti-pattern the deployment may still be useful. What it loses is reproducibility: auditors cannot reliably verify outcomes by replay, recovery teams cannot reconstruct state by re-applying changes, regression tests cannot isolate the contribution of rule changes from unrecorded variation, and migrations cannot reproduce substrate state on new infrastructure.

5. Operational decisions the composition forces

The architectural decisions §3 names translate to specific operational requirements a deployment must satisfy. Provenance is complete on every substrate change — all six fields, with prior-state and rule reference specifically populated. Change addresses are deterministic functions of provenance fields and survive replay. Cells depend only on substrate state and orchestration rule, with hidden state, implicit context, time, randomness, and external state outside bounded consultations excluded from substrate-write paths. Conflict resolution is cell-mediated under rules whose inputs are recorded in provenance. Allowed non-determinism — LLM consultation in particular — is bounded so that any effect on substrate state is either deterministic over recorded inputs or itself recorded in provenance, so that replay uses the recorded value rather than re-querying. Provenance is substrate-resident, not in external audit logs, because reproducibility is a substrate-only-path property. Prior-state is recoverable from any point in deployment lifecycle, either stored directly in provenance or derivable from substrate history.

6. What the composition is NOT

Four adjacent architectural patterns are commonly conflated with the composition. Each is a real and reasonable commitment in some other architecture; none is what the composition specifies.

Not snapshot-based recovery. Snapshot-based recovery restores substrate state from a backup. The deployment recovers state but does not replay history. The composition is broader: reproducibility supports replay of the sequence of operations, not just restoration of a single state. Snapshot recovery may complement reproducibility but does not substitute for it.

Not event sourcing. Event sourcing records events that produced state changes; some implementations are legitimate substrates under the source paper’s definition. The composition is more specific: it requires retraceability *and* determinism, not just event recording. An event-sourced system whose event handlers depend on hidden state, time, or randomness records events but cannot reproduce state on replay.

Not external audit logs. External audit logs record events outside the substrate. They may be useful for security, compliance, or operational forensics, but they are not substrate-resident. The composition forces reproducibility through substrate-only paths because the substrate is the source of truth; replay cannot depend on external systems whose retention, availability, or governance is outside the substrate’s. External audit logs may supplement reproducibility; they cannot substitute for it.

Not differential testing. Differential testing — running two versions of a system and comparing their outputs — operates *on* reproducibility outcomes; it is not itself the composition. The composition is what makes differential testing meaningful at the substrate layer; without reproducibility, two runs of the same version would already produce different results, and a differential test could not isolate the contribution of the version difference from unrecorded variation.

7. Operational test

A deployment satisfies the path retraceability $\times$ determinism contract composition if and only if all of the following hold at all times during the substrate’s existence:

1. Every substrate change carries all six provenance fields, with prior-state and rule reference specifically populated.
2. Change addresses are deterministic functions of provenance fields and survive replay.
3. Reads of substrate state from a known prior state produce identical results across executions.
4. Cell behavior depends only on substrate state and orchestration rule, with allowed non-determinism bounded to recorded inputs.
5. Conflict resolution is cell-mediated under rules whose inputs are recorded in provenance.
6. LLM consultations and other allowed non-determinism do not propagate to substrate state outside what provenance records.
7. Provenance lives in substrate; replay is drivable from substrate content alone.

Three sharpening properties make the test concrete for deployment review.

Replay-fidelity test. Select a historical substrate state. Replay the recorded provenance from a prior state up to that point. Compare the replayed state against the original. Mismatched replay results indicate composition failure — typically through cell-behavior non-determinism, hidden state, unrecorded time-dependence, or unbounded LLM consultation.

Addressable-provenance test. Select a sample of historical change addresses. Resolve each address to its provenance record. Verify each record carries complete fields sufficient for replay. Missing or inconsistent provenance indicates composition failure — typically through partial provenance, non-stable addressability, or external-log substitution.

Deterministic-substrate-operation test. Repeat a representative set of substrate operations — reads, writes, conflict-handling — over a fixed prior state. Compare results across repetitions. Non-deterministic outcomes outside the bounded non-determinism categories the contract permits indicate composition failure — typically through implicit context, hidden state, or unbounded external consultation.

A deployment that satisfies (1)–(7) and passes the three sharpening tests instantiates the composition. A deployment that satisfies one foundational commitment but not the other, or that satisfies both at a coarse grain but fails any of the sharpening tests at the operational grain, is using the term “reproducibility” for something the composition does not defend, and the difference should be named.

8. Why naming the composition as standalone matters

The composition is what makes substrate state architecturally audit-able and recoverable. Without publishing it as a standalone derivation, the reproducibility property is implicit in the foundational commitments rather than architecturally specified, and downstream readers are free to satisfy each commitment individually while failing the composition.

The failure modes the composition makes nameable are real ones in current practice. Implementations that record complete provenance whose replay produces different results give auditors a record they cannot rely on. Implementations whose operations are deterministic but unrecorded give recovery teams operations they cannot reconstruct. Implementations that resolve conflicts implicitly give replay a different history than the one that occurred. Implementations that pipe LLM outputs directly to substrate give substrate state a non-deterministic component that cannot be replayed. Each failure mode is identifiable only because the composition is stated; each is invisible if the foundational commitments are satisfied without their composition being verified.

This note is the sixth in Phase A4 of the derivation series. The five preceding compositions formalize governance-against-authority, governance-of-AI-through-rules, authority-locus-on-state-side, AI-mediated retraceability, and AI-mediated authority preservation. Together with reproducibility, these six establish foundational architectural patterns covering governance, AI-rule-mediation, authority-locus, AI-accountability, AI-substrate-authority, and substrate replay. Subsequent notes formalize additional architecturally significant compositions from the $C(16,2) = 120$ possible pairs of the foundational commitments.

Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should use “reproducibility” in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Composition Pair: Path Retracement × Determinism Contract — How Two Foundational Commitments Compose to Produce Reproducibility in CKS Substrates*. May 6, 2026. ORCID: 0009-0004-8065-3235.